

סיכום דברים חשובים למבחן – מבנה מחשב

גיא יער-און

Char=1 bytes,short=2,int,float=4,long=8,double=8,logn long=8

C:תזכורות של

א. & - לא להביא ערך של משתנה, אלא כתובת שלו בזכרון.

```
int a = 42;
```

```
printf("The value is %d\n", a); // מדפיס 42
```

```
printf("The address is %p\n", &a); // מדפיס כתובת בזיכרון (למשל 0x7ffd...)
```

ב. * - למצביע יש 2 תפקידים: בהגדרה הוא אומר "המשתנה הזה יאחסן כתובות", בשימוש הוא אומר "לך לכתובת שרשומה בתוך המצביע ותביא מה שיש שם"

```
int x = 10;
```

```
int *ptr = &x; // הגדרה: ptr שומר את הכתובת של x
```

```
*ptr = 20; // Dereference: "ושנה את הערך ל-20 ptr לכתובת ב"
```

```
printf("%d", *ptr); // ידפיס 20
```

ג. ** מצביע כפול: משתנה ששומר כתובת של משתנה אחר שהוא בעצמו מצביע. שימושי ליצירת מטריצות. כלומר ** זה כתובת אל מקום שמחזיק כתובת בעצמו.

תזכורת לתנאי בשורה אחת:

condition ? value_if_true : value_if_false

גלישות:

במספרים ללא סימן – גלישה מתרחשת כשיש נשא (carry) מהMSB. איך יודעים לזהות? כאשר חיברנו שני מספרים וקיבלנו תוצאה קטנה יותר מהם. למשל: $0 = 255 + 1$. כאן ידלק הCF

במספרים עם סימן – גלישה מתרחשת כשהתוצאה חורגת מהטווח האפשרי. איך נזהה? חיברנו שני מספרים עם אותו סימן, וקיבלנו סימן שונה. כאן ידלק הOF

	שהתעדכנו	Unsigned	
גלישת Signed: חיברנו שני חיוביים ויצא שלילי.	OF=1, SF=1, CF=0 $64 + 64 = -128$	$64 + 64 = 128$	+ 01000000 01000000
גלישת Unsigned: היה נשא (Carry), והתוצאה התאפסה.	CF=1, ZF=1, OF=0 $-1 + 1 = 0$	$255 + 1 = 0$	+ 11111111 00000001
גלישה כפולה: שגיאה בשתי הפרשנויות בו זמנית.	CF=1, OF=1, ZF=1 $-128 + (-128) = 0$	$128 + 128 = 0$	+ 10000000 10000000

דוגמה טובה:

מחרוזות נשמרות לפי הסדר הרגיל שלהן בזכרון – ללא ליטל אנדיאן.

לזכור בליטל אנדיאן: בתוך בייט בודד הביטים לא מתהפכים! רק הבייטים עצמם מתהפכים

כשמבצעים פקודת mov לרגיסטר של 32 ביט, המעבד מאפס אוטומטית את כל 32 הביטים העליונים של הרגיסטר המקביל לו (64 ביט). כשעובדים עם רגיסטר של 8 או 16 ביטים – הוא לא מאפס אוטומטית!

Sar – תזוזה אריתמטית ימינה, במקום להכניס תמיד אפס מצד שמאל המעבד משכפל את הביט השמאלי ביותר (ביט הסימן). למה זה נחמד? שומר על ביט הסימן – אם היה שלילי ישאר שלילי גם אחרי הפעולה. זה טוב לעבודה עם signed.

חשוב:	פקודה	כיוון	מה נכנס מהצד הריק?	שימוש עיקרי
	SHL / SAL	שמאלה	אפסים (0)	כפל בחזקות של 2
	SHR	ימינה	אפסים (0)	חילוק מספרים ללא סימן
	SAR	ימינה	העתקה של ביט הסימן	חילוק מספרים עם סימן

Performance זה "לעשות יותר" ב"פחות זמן" – כמה שפחות פעולות. יעילות.

אדרסינג מוד: האמצעי חייב להיות רגיסטר ואסור שיהיה rsp (חומרית לא אפשרי). השמאלי (בייס) יכול להיות כל רגיסטר.

אם יש לנו cmpq %rax,%rbx אזי:

1. Rax>rbx אם ורק אם SF!=OF.

2. Rax<rbx אם ורק אם SF=OF.

פעולת test – רק משנה את ערכי הדגלים, לא משנה ערכי רגיסטרים!

פקודות set: פועלת תמיד על רגיסטר 8 ביט בודד, בודקת תנאי מסויים על הדגלים ואם התנאי מתקיים היא שמה 1 ביעד, ואם לא היא שמה 0. היא לא משנה את שאר הרגיסטר! למשל sete שמה 1 ברגיסטר אם zf=1. חשוב לשים לב להוסיף movzbl כי היא משנה ערך 8 ביט בודד – ויתכן שאנחנו מחזירים ערך גדול יותר לכן נאפס את השאר. מאפשר לקרוא את ערכי הדגלים מבלי לשנות אותם.

הסיבה שבארדסינג מוד אפשר רק 1,2,4,8 היא כי אלו הגדלים הסטנדרטיים של טיפוסים נתונים בסיסיים בזכרון והחומרה מותאמת לאפשר קפיצות במרחקים שתואמים בדיוק לגדלי הטיפוסים הללו שיש בסי על מנת לייעל גישה למערכים וכו'.

יעד של פקודה חייב להיות רגיסטר/זכרון – לא אימדיאט

פקודת lea לא ניגשת לזכרון ולא משנה ערכי דגלים

אפשר לחשב מודלו בצורה הבאה: עושים חילוק וזה מחשב גם את המנה ושומר אותו בrax ואת השארית בrdx בו זמנית.

שלב lodaing: מערכת ההפעלה מעלה את התוכנית לזכרון. ע"י loadern. הוא מקצה מרחב כתובות (יוצר ram את כל מפת ה"חלקים" השונים כמו ביאסאס דטה וכו'), מאתחל bss: רק בשלב זה סגמנט bss מתאפס ל0 בפועל, יוצר מחסנית heap, דוחף למחסנית את argv,argc, מעדכן את rip לכתובת של פונקציית _start.

על החלקים השונים של תוכנית:

- קובץ האובג'קט שנוצר לאחר שלב האסמבלר לא יכול לרוץ כשלעצמו כתוכנית עצמאית כיוון שכתובות של פונקציות + משתנים חיצוניים עוד לא ידועות.

אופרנד באסמבלי	מה המעבד עושה?	מקביל ב-C
<code>a\$</code>	משתמש בערך הכתובת של המשתנה <code>a</code> .	<code>a&</code> (כתובת המשתנה)
<code>a</code>	ניגש לכתובת של <code>a</code> ומביא את התוכן.	<code>a</code> (ערך המשתנה)
<code>\$10</code>	משתמש במספר 10 כערך קבוע.	<code>10</code> (Literal)
<code>rax%</code>	משתמש בערך שנמצא בתוך הרגיסטר.	<code>x</code> (משתנה ברגיסטר)
<code>(rax%)</code>	הולך לכתובת שבתוך הרגיסטר ומביא תוכן.	<code>*p</code> (Dereference)

חשוב! אין דבר כזה `$register`!

`JumpTable`: רציף בזכרון. אם הקייסים שלנו לא רציפים אפשר להשתמש ב`rept` – זה שם את הערך הדיפולט בכל אחד מהם (הוא מזהה את ערך הדיפולט שהרי נמצא בסוף). זה לא יעיל כמובן! הקומפיילר ישקול היטב מתי ג'מפ טייבל ומתי לא.

תחביר כמו `mov 8, rax` (בכוונה 8 לבד ללא כלום!) זה אומר לגשת לכתובת זכרון 8 – חוקי, אך לא נפוץ.

אי אפשר לבצע פעולות כמו `cmp $8, $16`: לא חוקי. כמו שאי אפשר עם שני ערכים מהזכרון.

כשעושים `test %rax, %rax`: זה בשביל לבדוק את הסימן של הערך ברגיסטר / האם הוא אפס.

- ZF ידלק אם `rax=0`

הבדל בין פונקציות לבין סתם ג'אמפ: כשעושים פונקציה מובטח לנו כי נחזור למקום שנקרא ממנו. בג'אמפ – לא. לשים לב לזה!

- כשעושים `call function` זה קודם כל מוריד את ערך `rsp` ב8, ואז דוחף את ערך החזרה. (8 כי כתובת).
- חשוב! הפקודה `globl main` מציינת שהתווית `main` היא "ציבורית", ובכך מאפשרת ל-Linker למצוא אותה מחוץ לקובץ הנוכחי ולהגדיר אותה כנקודת הכניסה לתוכנית.
- פקודת `nop` – פקודה שלא עושה כלום, משמשת ליישור קוד בזכרון או להשהיות קטנות. בחומרה היא ממומשת ע"י החלפה של הרגיסטר עם עצמו – לכן היא כן לוקחת סייקלים של `fetch, decode, execute`
- מדוע משתמשים ב`xor %rax, %rax` במקום `movq $0, %rax`? הקידוד של `xor` קצר יותר בבתים ומהיר יותר לביצוע בחלק מהמעבדים – תופס פחות מקום בקאש של אינסטרקשנס

חשוב!

פקודה	האם חוקי?	מה זה עושה בפועל? (מתחת למכסה המנוע)	הסבר פשוט
<code>jmp label</code>	כן	<code>RIP = address_of_label</code>	קפיצה ישירה: פשוט רץ לכתובת של התווית שכתבנו בקוד.
<code>jmp %rax</code>	לא	--	שגיאת סינטקס: אסור לקפוץ "לתוך" רגיסטר.
<code>jmp *%rax</code>	כן	<code>RIP = %rax</code>	קפיצה עקיפה (רגיסטר): המעבד לוקח את הערך שבתוך <code>rax</code> ושם אותו ב- <code>RIP</code> .
<code>jmp *(%rax)</code>	כן	<code>RIP = M[%rax]</code>	קפיצה עקיפה (זיכרון): הולך לכתובת שרשומה ב- <code>rax</code> , שולף משם כתובת אחרת, וקופץ אליה.

- הלינקר אחראי על החלפת שמות המשתנים והפונקציות בכתובות זכרון סופיות/ יחסיות.

תהליך הקומפילציה:

1. Preprocessor – שלב של עריכת טקסט בלבד. מחפש סימנים שמתחילים ב # ו: 1. מחליף מאקרואים (define), 2. כל include פשוט מעתיק ומדביק את תוכן כל הקובץ שמרחיבים לקובץ הנוכחי. 3. מוחק את כל ההערות מהקובץ. פלט: קובץ טקסט זמני בסיומת i.
2. Compiler – זה השלב של ה"מוח". הופך קוד בשפה גבוהה לשפה שקרובה לחומרה. 1. בודק סינטקס חוקי. 2. אופטימיזציה – משפר את הקוד בהתאם. 3. תרגום לאסמבלי – מתרגם את הקוד לאסמבלי. פלט: קובץ אסמבלי בטקסט בסיומת s.
3. Assembler – השלב בו הטקסט הופך לבינארי (01), 1. תרגום לOpcodes: הופך פקודות לקודים בינאריים שהמעבד מבין. 2. יוצר elf – בונה בלוקים של זכרון (טקסט, דטה וכו'). 3. יוצר symbol table – רושם את כל השמות שהגדרנו. 4. יוצר relocation table (עבור כל ה"חורים" שאיננו יודעים לטפל כעת). פלט: קובץ אובייקט בינארי בסיומת o.
4. Linker – מחברים את כל ה"חתיכות" לתוכנית אחת גדולה. 1. בשלב זה יש את פתרון הסמלים – לפי החוקים שלמדנו. 2. לוקח את כל קבצי הס ומאחד את הטקסט שלהם לבלוק אחד ואת הדטה שלהם לבלוק אחד. 3. מוסיף קוד ספריות. פלט: קובץ הרצה סופי לרוב a.out.
5. Loading – זה קורה בעת שאנחנו מריצים את התוכנית - a.out. - כאן מערכת ההפעלה נכנסת לתמונה. 1. מקצה זכרון (יוצרת מרחב וירטואלי של התוכנית ב ram), 2. העתקה של הקוד והדטה ל ram. 3. מאתחלת מחסנית וערימה. 4. מאתחל את rip לנקודת ההתחלה של התוכנית _start. 5. מעביר את השליטה למעבד כדי שיתחיל לבצע פקודה ראשונה.

תהליך הלינקינג:

- Listing file – קובץ טקסט שנוצר במהלך שלב האסמבלר, "מסמך העל" שמאפשר למתכנת לראות בדיוק כיצד האסמבלר תרגם את הקוד שלו לשפת מכונה. הקובץ מחולק לעמודות שמראות כתובת יחסית (מיקום פקודה מתחילת סקשן), קוד מכונה (הערכים הבינאריים של הפקודות והנתונים), מספר שורה.
- Symbol Table – נכנסים לטבלה פונקציות (גלובליות וסטטיות), משתנים גלובליים ומשתנים סטטיים. לא נכנסים – לוקליים. רושמת מה יש לה ומה חסר לה.
- 1. Undef – מגדירים זאת כי לא הגדרנו את זה בקובץ. סימן ללינקר לחפש בקבצים אחרים.
- 2. Global – קבצים אחרים יכולים לקרוא לה
- 3. Extern – מוגדר בקובץ אחר הפונקציה שלנו ואנחנו נקשר אותה בהמשך בלינקינג

- Relocation Table - הטבלה סימבול אמרה מה חסר, הטבלה רלווקיישן אומרת *איפה* חסר ואיפה צריך לתקן. אומר איפה למלא את החורים בעתיד.
- 1. Abs – כתובת מוחלטת בזכרון (ידועה כבר). אם סימבול הוא abs אזי שהוא משתנה/ערך
- 2. Rel – הכתובת היחסית. זה לא הכתובת האמיתית אלא מרחק בין פקודה נוכחית ליעד. אם סימבול הוא rel אזי שהוא פונקציה.
- Elf relocatble format - קובץ בינארי שנוצר על ידי האסמבלר (קובץ o) המכיל קוד ונתונים שמאורגנים בסקשנס אך כתובות הזיכרון שלו עדיין לא סופיות.

ספריות סטטיות ודינאמיות:

- סטטיות: הלינקר לוקח את קוד הספרייה ופשוט מעתיק אותו לתוך קובץ ההרצה הסופי שלי. הלינקר מחפש בזמן לינקוף את הפונקציות השתמשנו בהן ומעתיק אותן ל[a.out](#)
- 1. יתרונות – עצמאיות, התוכנית תרוץ על כל מחשב גם אם הספרייה לא מותקנת שם. מהירות – אין צורך לחפש ולטעון ספריות בזמן ריצה כי הכל בזכרון.
- 2. חסרונות – שכפול קוד, אם יש באג אין אצלנו גרסה מעודכנת מיידי.
- דינאמיות: מועלות בזמן ריצה, הקוד לא מועתק לתוכנית שלנו. הלינקר רק שם הערה שאומרת – "התוכנית הזו צריכה ספרייה איקס". החיבור האמיתי קורה רק כאשר המשתמש מריץ את התוכנית, ה [loader](#) טוען את הספרייה לזכרון רק פעם אחת וכל התוכניות שרצות משתמשות באותו עותק
- 1. יתרונות: חיסכון בזכרון, קל לעדכן באגים ללא צורך לקמפל מחדש
- 2. חסרונות: בזמן ריצה, כי לוקח זמן לחפש איפה הפונקציה יושבת בזכרון.

:PIC

- הלינקינג מחשב את ה offset כבר בזמן הלינקוף וצורב אותו בפקודת מכונה.
- ספריה דינאמית חייבת להיות PIC, קובץ הרצה רגיל (executable) לא חייב להיות
- המרחק בין הטקסט לדטה חייב להיות קבוע.
- חרון של PIC – לחשב את החישוב המתמטי של ה offset לוקח זמן.
- ערך של lable הוא תמיד הכתובת שלו! \$ נותן ערך. ולכן \$str יתן את ערך הכתובת של str.
- ASLR – טכניקת אבטחה של מערכת ההפעלה שנועדה למנוע התקפות סייבר. בכל פעם שמריצים את התוכנית, ה [loader](#) מגריל כתובת בסיס אחרת בזכרון עבור חלקי התוכנית השונים (סטאק, היפ, ספריות דינאמיות וכו'). מתחילים בכתובות אקראיות. בלי PIC לא היה אפשר להשתמש בזה ביעילות – כי אם הקוד היה אבסולוטי, הלאודר היה חייב לטעון את התוכנית בכתובות האבסולוטיות שלהם ואם היה מגריל כתובת אחרת התוכנית הייתה קורסת.

בוא נבין מה קורה כשיש לנו `str$` (אבסולוטי):

1. בזמן הלינקוף (Link-time): הלינקר מסתכל על הקוד ואומר: "אוקי, החלטתי ש- `str` יושב בכתובת 2030". הוא לוקח את הפקודה `movq $str, %rdi` ופשוט צורב לתוכה את המספר 2030. הקוד הבינארי עכשיו מכיל את המספר 2030 בצורה קשיחה.
2. בזמן הטעינה (Load-time): מגיע ה- [Loader](#) "המבולבל" מהשקופית. הוא מחליט לשים את הנתונים בכתובת 3030.
3. בזמן הריצה (Run-time): המעבד מריץ את הפקודה הצרובה. הוא רואה: "שים ב- `rdi` את המספר 2030". הוא לא יודע שזאת! הוא פשוט מבצע מה שכתוב לו בקוד. הוא ניגש ל-2030, אבל שם כבר אין כלום, כי הכל זז ל-3030.

חשוב! כל אחד והתפקיד שלו, וזה למה pic טוב אם יש [loader](#) מבולבל - הלאודר לא תמיד מקשיב לכתובות שהלינקר תכנן) במיוחד בגלל ASLR או ספריות דינאמיות).

- אם ננסה להריץ תוכנית שתלויה בספריה דינאמית שנמחקה מהדיסק, התוכנית תכשל בטעינה ותציג שגיאה: loader ינסה למצוא את הקובץ לפי שמו, ואם לא ימצא לא יוכל להשלים את פתיחת התוכנית.
- עבור תהליכים שמשתמשים באותה ספריה דינאמית, יש להם text section משותף

הערה חשובה על lazy binding: הקובץ (.so) נטען ל RAM-לפני שה main-מתחיל, אבל הכתובת הסופית של הפונקציה בתוכו נפתרת רק ברגע שקוראים לה.

מושגים חשובים:

- ספריה היא אוסף של קבצי o
 - Load time – בזמן טעינת התוכנית לזכרון מיד הקוד שלנו טוען יחד איתו את הדינמיק לינקר שקורא את כל הספריות החיצוניות בהן הקוד שלנו ישתמש, הן נטענות לזכרון יחד עם התוכנית שלנו.
 - Run time – לא נטען ישר את כל הספריות לזכרון, אלא נחכה לפעם הראשונה שנראה סימבול בעת הרצת התוכנית ואז הלינקר יטען את הסימבול הנוכחי בזכרון.
 - Lazy binding – דינאמיק לינקינג + RunTime .
 - GOT: טבלה שנמצאת בדטה סקשן (לכן היא פרטית לכל תהליך! וניתנת לכתיבה). תפקידה להחזיק את הכתובות האמיתיות ב RAM של פונקציות ומשתנים גלובליים חיצוניים. הקוד של pic תמיד יגש לגOT באמצעות אופסט יחסי לRIP. (כי מרחק בין דטה לטקסט קבוע).
 - PLT: טבלה שנמצאת בטקסט (משותפת לכולם). תפקידה לשמש כמתווך עבור קריאות לפונקציות.
- בפעם הראשונה שקוראים לפונקציה בלייזי ביידינג –

1. התוכנית קוראת לplt@printf (לדוגמה) – קודם כל plt
2. קפיצה לגOT – הPLT מבצע קפיצה לכתובת שרשומה בGOT עבור printf .
3. בפעם הראשונה, הכתובת בgot לא מצביעה לprintf אלא מצביעה חזרה לשורה הבאה בplt
4. plt דוחף מזהה של הפונקציה למחסנית וקורא לדינמיק לינקר
5. הלינקר מוצא את printf בRAM וכותב את הכתובת האמיתית שלה בgot
6. כעת, בכל פעם שנקרא לפונקציה שוב הקפיצה לגot תוביל אותנו ישירות לפונקציה האמיתית בספרייה.

כך זה נראה תרשימית (מסיכום אופק 2026, תודה!)



- got לא משותף בין תהליכים שונים שמשתמשים באותה ספרייה – הקוד (טקסט) משותף אך got נמצא בdata שייחודי לכל תהליך. לכל תהליך יש got משלו!
- מדוע צריך patching ומדוע נשתמש בסop במקום פשוט למחוק שורות מהקוד? Patching נועד לערוך את הקובץ הבינארי ולשנות את התנהגות התוכנית בעזרתו, גם מבלי שיש לנו את קובץ הc המקורי. אסור למחוק סתם שורות בייטים מהקוד – כיוון שכל הנתונים מבוססים על אופסטים ואם נוסף / נוריד בייט – כל הכתובות ישתנו. לכן נרצה להשתמש בסop שלא עושה כלום אך תופסת מקום בשורה.
- Plt משותף לכל התהליכים.

:Cache

- עקרון הלוקאליות – אם משהו נמצא בסביבת מה שאני נמצא בו (X , Y לידי), בסבירות גבוהה שנמצא להשתמש גם ב- Y .
- Temporal locality – אם השתמשתי במשתנה, בסבירות גבוהה שאשתמש בו שוב.
- Spatial locality – אם יש משתנה / פקודה לידי בזכרון, בסבירות גבוהה רצו (המתכנת) שיהיה שימוש בפקודה/משתנה זה.
- גישה לקאש הרבה יותר מהירה מגישה לזכרון (פי מאה בערך).
- התהליך למציאת ערך x כלשהו:
 1. בדיקה אם x ב- $L1$. אם כן, נביא אותו אל ה- cpu רגיסטר.
 2. בדוק אם x ב- $L2$. אם כן, נביא אותו אל ה- cpu רגיסטר. נרצה להעביר אותו גם אל ה- $L1$.
 3. אחרת, נקרא את x מה- Ram .
- LRU – מפנים את השורת קאש שלא נגענו בה הכי הרבה זמן.
- LFU – מפנים את שורת הקאש שניגשו אליה הכי פחות פעמים.

מבנה הקאש:

- קאש הוא מערך של קבוצות. כל סט מכילה מס' של שורות.
- נסמן את הסימונים הבאים:
 1. S – מס' הקבוצות. מתקיים $2^s = S$ כאשר s הוא מס' הביטים שנדרשים לייצוג S קבוצות.
 2. E – מס' השורות בכל set .
 3. B – גודל כל שורה. מתקיים $2^b = B$ כאשר b הוא מס' הביטים שנדרשים לייצוג b ביטים – של $offset$ בתוך השורה. (שורה=בלוק)

חישוב מתמטי של כתובת:

1. בתחילה נחשב $s = \log(S)$, $b = \log(B)$ ואת $t = m - (s + b)$ כאשר m הוא אורך הכתובת.
2. הכתובת תחולק ככה: $|offset (b \text{ bits})| |Index (s \text{ bits})| |Tag (t \text{ bits})|$

סוגים של cache ויתרונותיהם:

1. **Direct-Mapped**: $E=1$, כלומר יש שורה אחת בכל קבוצה. כלומר כל כתובת מהזכרון יכולה להכנס לשורה אחת בלבד בקאש.

יתרונות: קל לחפש, פשוט נסתכל על tag . זול יחסית.

חסרונות: conflict miss – אם יש לנו 2 נתונים שהתוכנית ניגשת אליהם הרבה, הם יעיפו אחד את השני כל הזמן גם אם שאר הקאש ריק לחלוטין.

2. **Fully Associative**: $S=1$. אין אינדקס, כל כתובת יכולה להכנס לכל שורה פנויה בקאש. יש סט אחד בדיוק.

יתרונות: אין כמעט conflict miss! נתון יזרק מהקאש רק אם הוא באמת יתמלא.

חסרונות: איטי מאוד – בשביל לחפש נתון יש להשוות את tag שלו לכולם.

3. **K-Associative-Mapped**: $E, S > 1$: דרך הביניים (מה שיש ברוב המעבדים), הקאש מחולק לסטים ובכל סט K שורות.

יתרונות: מפחית קונפליקטים בהשוואה לדיירקט מאפ, ומפחית את זמן החיפוש ביחס לפולי אסוסיאטיב. כלומר: פשרה טובה בין שניהם.

חסרונות: יותר יקר לחיפוש בהשוואה לדיירקט מאפ, יותר קונפליקטים בהשוואה לאסוסיאטיב.

סוג קאש	S (סטים)	E (שורות לסט)	סיבוכות חיפוש	Conflict Misses	סיכום:
Direct	הרבה	1	הכי נמוכה ($O(1)$)	גבוהה מאוד	
K-Way	בינוני	K (למשל 8)	בינונית	נמוכה	
Fully	1	הכל	גבוהה ($O(E)$)	אפסית	

- בתחילת הריצה כל ה-bit valid הם 0.

סוגי miss:

1. COLD MISS – מיס שנגרם כי הקאש עדיין לא הכיל את המידע. תמיד יהיה בתחילה קולד מיס!
2. Conflict miss – מצב של דריסה, יש בלוק מהזכרון שמגיע והוא דורס מידע אחר. ישנם שני נתונים שמתחרים על אותו מיקום. הקוד אינו קאש פרינדלי.
3. Capacity miss – הקאש קטן מדי בשביל להכיל את כל המידע שאני צריך. אין מה לעשות איתו.

- קביעת ה-set:

1. אם היינו לוקחים את הביטים הימניים ביותר לייצוג הסט'ס (ולא האמצעיים) – לא טוב, דברים שיופיעו אחד אחרי השני בקוד יהיו בסטים שונים מה שיפגע בlocality principles
2. אם היינו לוקחים את הביטים השמאליים ביותר לייצוג הסט'ס (ולא האמצעיים) – לא טוב, יהיו הרבה כתובות עד שהשמאליים יתחלפו ויווצר מצב של עומס של המון כתובות שנכנסות לאותו סט מה שיצור עומס אדיר ולכן התנגשויות.

- Write hit**: הנתון כבר בתוך הקאש. מתי מעדכנים את הזכרון הראשי?

1. **Write-through**: כותבים את הערך החדש לקאש ומיד גם לרמה שמתחתיו (לזכרון או L2)

יתרון: הזכרון הראשי תמיד מעודכן. חסרון: כתיבה לזכרון היא איטית.

בשיטת **Write-through** המעבד מעדכן את ה-RAM בכל כתיבה. מכיוון שה-RAM איטי מאוד, המעבד נאלץ לבצע **Stall** (עצירה) ולחכות שהכתיבה תסתיים. זה פוגע קשות בביצועים.

הפתרון: Write Buffer:

הפתרון הוא להוסיף רכיב זיכרון קטן ומהיר בין המעבד ל-RAM שמתפקד כתור (Queue).

איך זה עובד? כשיש פקודת כתיבה, המעבד שולח את הנתון + הכתובת ל-Buffer וממשיך מיד לפקודה הבאה. הוא לא מחכה ל-RAM.

העבודה של ה-Buffer: הוא מחזיק את הבקשות (בדרך כלל 1-8 בקשות) ומעביר אותן ל-RAM ברקע, בקצב שה-RAM מסוגל לקבל.

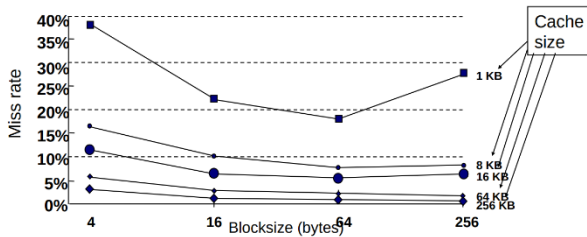
2. **Write-back**: כותבים את הערך החדש רק לקאש כרגע, העדכון לזכרון הראשי נדחה עד לרגע שבו נצטרך להעביר את השורה הזו כדי לפנות מקום לערך אחר. בשלב זה נבין מדוע צריך דירטי-ביט.

- Write-Miss**: הנתון לא בקאש. האם להתאמץ ולהביא את הבלוק מהזכרון רק בשביל לכתוב לתוכו?

1. **Write-Allocate**: טוענים את הבלוק מהזכרון לתוך הקאש, ואז מעדכנים אותו בתוך הקאש. למה זה טוב להביא את הבלוק? כי אם אתה הולך לשנות את המשתנה עוד 100 פעמים בקרוב, כדאי שיהיה קרוב אליך בקאש ושלא תצטרך לדבר עם הזכרון בכל פעם.

2. **No-Write-Allocate**: כותבים ישירות לרמה שמתחתיו (לזכרון) בלי להכניס את הבלוק לקאש בכלל. הבלוק ישאר בזכרון עד לביצוע פעולת קריאה.

חשוב: נסתכל על הגרף הבא –



- ככל שנעים על ציר האיקס מ 4 ל 64 – אחוז הפספוסים יורד. למה? כי בלוק גדול מנצל טוב spatial locality. כשנביא בלוק גדול – נביא על הדרך הרבה שכנים, אם התוכנית תגש אליהם אחרי הנתון שלי יהיה פחות פספוסים.
- אז למה לבסוף הגרף עולה? נבחין כי בבלוק של 256 בתים, אחוז הפספוסים עולה חזרה. הסיבה: אם הבלוקים גדולים מדי, נכנסים פחות בלוקים שונים לקאש בו זמנית. נתונים יריבו על המיקום בזכרון מה שיגרום לזה שיעיפו הרבה אחד את השני – conflict miss.
- להזכר בנוסחה $C=BES$ באשר C זהו גודל הקאש הכולל. אפשר להסיק על מה יקרה כשמגדילים/מקטינים את אחד המשתנים

Ram-Structure:

- Cpu לא נוגע פיזית בזכרון. הם מתקשים באמצעות חוטים שנקראים Bus.
- Address Bus – CPU שולח את הכתובת (היכן הנתון?)
- Data Bus – כאן עובר המידע עצמו (מהו הנתון?)
- Control Bus – כאן עוברות הפקודות של "תקרא"/"תכתוב".
- קריאה: שם את הכתובת ב Address Bus, מאפשר קריאה, מוציא את המידע דרך הדטה באס.
- כתיבה: שם את הכתובת ב Address Bus, שם את המידע לכתיבה בדטה באס, מאפשר כתיבה.
- טרנזיסטור: רכיב חשמלי שהוא בסך הכל מפסק. אם יש זרם בשן האמצעית, הזרם עובר (ביט 1), אחרת ביט 0. טרנזיסטורים משולבים בונים שערים לוגיים.
- Sram: כל ביט מיוצג ע"י 6 טרנזיסטורים – בגלל שהחשמל נוטה להחלש והם מקטינים את הירידה בחשמל. הוא הרבה יותר יקר מDram! הוא גדול יותר גם פיזית (נפח גדול יותר)
- Dram: כל ביט מיוצג ע"י 2 טרנזיסטורים – איטי הרבה יותר, ולכן זול הרבה יותר.
- Cache עשוי מטכנולוגיה של Sram: לכן הוא קטן, ולכן הוא ממש יקר. נרצה שיהיה כמה שיותר קטן – בשביל שיהיה מהיר מאוד.
- Buffer Overflow: מהו למעשה? חוסר בבדיקת גבולות למשל (שפת סי לא מבצעת בדיקה האם הגישה למערך חורגת מהגבולות שלו), אחסון במחסנית – משתנים מקומיים נשמרים על המחסנית יחד עם מידע קריטי של המערכת כמו ערכי רגיסטרים וכתובות חזרה, השחתת זכרון – שילוב של חוסר בבדיקת גבולות + אחסון במחסנית: מאפשר כתיבה מחוץ לגבולות המערך לדרוס ולהשחית את המידע ששמור במחסנית. קריסת התוכנית – כאשר התוכנית מנסה לטעון מחדש את הרגיסטרים או לבצע פעולת ret: היא עלולה להתנהג בצורה בלתי צפויה ולקרוס.

אופטימיזציות

Pipeline:

1. Fetch – מביא את הפקודה הבאה לביצוע מהזכרון או מהקאש, שמים את הכתובת שיש בRIP בAddress Bus, יהיה מהיר כמובן יותר אם הפקודה כבר בקאש.
 2. Decode – לוקח את ISA ובודק אם הפקודה חוקית או שלא, כלומר מפענח את הפקודה.
 3. Read – לא יתבצע תמיד. לא לכל פקודה יש קריאה מהזכרון, בכל אופן: קריאה מהזכרון.
 4. Execute – ביצוע הפקודה.
 5. Write – לא תמיד יתבצע. אם עלינו לכתוב משהו חזרה לזכרון/קאש – יקרה בשלב זה.
- שלבים אלו בלתי תלויים אחד בשני, ולכן נוכל לבצע אותם בצורת "מדרגה".

פקודה	שלבים בשימוש	מה קורה בכל שלב?	הערות למבחן
ALU (add, mov reg,reg)	F, D, R, E, W	קריאת פקודה, פענוח, קריאת רג'יסטרים, חישוב ב-ALU, כתיבה לרג'יסטר.	הפקודה הכי "סטנדרטית".
Load (mov mem,reg)	F, D, R, E, W	ב-E מחושבת הכתובת, ובשלב ה-W (או שלב זיכרון משולב) הנתון חוזר.	הכי מסוכנת: פקודה שבאה אחריה וצריכה את הנתון הזה תיתקע ב-Stall כי הנתון מוכן רק בסוף.
Store (mov reg,mem)	F, D, R, E	קריאה, פענוח, קריאת רג'יסטר שרוצים לשמור, חישוב כתובת ב-E וכתיבה לזיכרון.	אין W: היא לא כותבת בחזרה לרג'יסטרים, אז שלב ה-W ריק.
Branch (jne, jmp)	F, D, R, E	קריאה, פענוח, קריאת רג'יסטרי השוואה, וביצוע השוואה ב-E.	כאן מתקבלת ההחלטה: ב-E אנחנו יודעים אם הניחוש היה נכון.

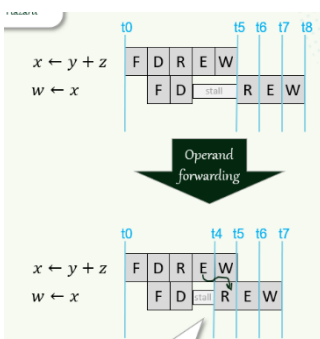
- **בהינתן הפקודות הבאות, אילו שלבים יתבצעו?**

- בפקודות השוואה כמו jne,jnz,je וכדומה – מתבצע הכל פרט לW.

- **Operand Forwarding:** נתבונן בדוגמה משמאל להמחשה. מדובר בקיצור שמאפשר לנו להשתמש בתוצאה של פקודה עוד לפני שהיא נכתבה פיזית בחזרה לרג'יסטר. כשיש לנו שתי פעולות שתלויות זו בזו, ללא forwarding הפקודה השנייה חייבת לחכות עד שהראשונה תסיים את שלב Write. אך – המעבד מבין כי התוצאה כבר מוכנה בסוף שלב Execute, היא אולי עוד לא נכתבה לרג'יסטר (זה יקרה בסוף השלב החמישי), אך היא קיימת פיזית בALU. ולכן: איך זה עובד? המעבד מוסיף נתיב עקיפה, שלוקחים את התוצאה ישירות מהמוצא של שלב E ומוזריקים אותה ישירות לכניסה של הR (או Rn) של הפקודה הבאה. עדיין צריך את W! אך הפקודה הבאה לא צריכה לחכות לW!

- **Branch prediction:**

בגלל הפייפליין, המעבד מביא (Fetch) את הפקודה הבאה עוד לפני שהוא סיים לכתוב את הפקודה הנוכחית. כשיש פקודת תנאי, כמו jne וכו', המעבד לא יודע אם לקפוץ לכתובת אחרת או להמשיך לפקודה הבאה, עד



שהוא יגיע לשלב Executen. ולכן: המעבד מנחש. ניחוש נכון – הוא ימשיך לזרום ללא הפרעה. ניחוש שגוי – המעבד מגלה את הטעות בשלב Execute. בשלב זה, פקודות Fetch, Decode כבר זיהמו את פייפליין בפקודות לא רלוונטיות. תוצאת הטעות: המעבד מבצע Flush: מנקה את הפייפליין ומתחיל מחדש מהכתובת החדשה. למעשה הוא ביצע אותם לחינם. ישנם כמה סוגים שונים:

1. **Static Branch prediction**: חיזוי סטטי, חיזוי שנקבע מראש בלי קשר למה שקורה בזמן ריצת התוכנית. הכי פשוט וזול למימוש חומרתי. ישנם שני סוגים:
 - BackWards (קפיצה לאחור): המעבד מניח כי נבצע את הקפיצה. זה קורה לרוב בלולאות – ולכן הניחוש הזה לרוב צודק.
 - Forwards (קפיצה קדימה): המעבד מנחש שלא נבצע את הקפיצה, זה קורה לרוב בif שבו אנחנו פשוט ממשיכים הלאה לקוד הבא.

חסרון שלו: לא מסתגל לשינויים, אם יש תנאי שמתנהג הפוך מהניחוש הסטטי נחטוף הרבה stalls.

2. **Dynamic Branch Predictor**: חיזוי דינמי. כאן המעבד הופך להיות "לומד". הוא משתמש בהיסטוריה של הריצה על מנת לדמות את התוכנית. המעבד מחזיק טבלה קטנה בזכרון, לכל פקודת קפיצה בתוכנית הוא רושם לעצמו מה קרה בפעמים האחרונות שהגענו אליה.

בשלב Executen המעבד מגלה אם הוא צדק בניחוש או שטעה. (בשלב זה מבצע ALU את ההשוואה ומגלה).

- **Random Branch Predictor** – מנחש באופן רנדומלי אם לקפוץ או שלא.

- **Conditional move (cmov)**: הרעיון ב CMOV הוא לחשב את שתי האפשרויות מראש, ורק בסוף לבחור את הערך הנכון. מדוע זה טוב? אין קפיצות לכן אין ניחושים, והפייפליין ישאר מלא ללא פלאשים. מבנה הפקודה: `cmovCC source,dest` כאשר CC יהיה התנאי (כמו e, ne, g, l וכו').

מתי לא נבצע זאת? בשלושת המקרים הבאים:

1. שני החישובים הם קשים לביצוע, עדיף להסתכן בלהמר מאשר לחשב את שניהם.
2. אם החישוב יגרום לשגיאה של זמן ריצה אם נבצע אותו ללא בדיקת התנאי – כמו גישה לnull
3. אם מדובר בשני חישובים שמשנים ערכים של משתנים.

- **Loop unrolling** – ברור מה זה. מתי לא נעשה?

1. גורם לניפוח קוד ופוגע בinstruction cache! אם הלולאה הפתוחה גדולה מדי, יתכן שלא תכנס לקאש של המעבד. ולכן במקום להריץ לולאה שיושבת בזכרון המהיר קאש נגש לזכרון הראשי בכל איטרציה – זה כמובן לא טוב!
2. אם זנב הלולאה מבזבז יותר מדי – כי זה לא מתחלק בערך שעשית לו אנרלוינג.
3. (אולי) סכנה לבלבל את הבראנץ' פרדיקטור.

- מה הקומפיילר לא יודע לעשות / עושה בעייתי?

1. לזכור שב-03 כמו $c < c+1$ מוחלפים ב `while(true)` ויכולים ליצור בעיות של לולאות אינסופיות למרות שלא כיוונו לכך במקור.
2. הוא לא מוציא החוצה `strlen`
3. לא יבצע אופטימיזציה משמאל: כי עם $x=y$ נקבל שהקודים אינם שקולים.

```
void f(int *x, int *y) {
    *x += *y;
    *x += *y;
}
```



```
void f(int *x, int *y) {
    *x += 2 * (*y);
}
```

```
void compute_total_dest_naive(int* vec, int len, int* dest) {
    *dest = 0;
    for (int i = 0; i < len; i++)
        *dest += *vec++;
}
```

4. Memory aliasing: נתבונן בדוגמת הקוד הבאה.

```
void compute_total_dest_buffer(int* vec, int len, int* dest) {
    int tot = 0;
    for (int i = 0; i < len; i++)
        tot += *vec++;
    *dest = tot;
}
```

במקום לגשת לזכרון בכל שלב, נוכל לכתוב את התוכנית הבאה:

הקומפיילר לא יבצע את האופטימיזציה הזו. מדוע?

Memory Aliasing הוא מצב בו שני פוינטרים שונים מצביעים על אותה כתובת בזכרון. זו הסיבה המרכזית לכך, שקוד שנראה לנו פשוט לא עובר אופטימיזציה באופן אוטומטי. הקומפיילר לא יכול להניח כי `dest, vec` מצביעים למקומות שונים. אחרת, בתרחיש האימה שיתואר מטה, סכום המערך יסכום גם את האיבר השלישי, שכבר השתנה, ולא נקבל תוצאה נכונה.

תרחיש האימה (Memory Aliasing):

נניח שמישהו קרא לפונקציה כך ש- `dest` מצביע בדיוק לאחד האיברים בתוך המערך `vec`. לדוגמה: `compute_total_dest_naive(vec, 5, &vec[2])`

- **בגרסה ה-naive:** כשנגיע לאיטרציה שבה המערך עובר על האיבר השלישי, אנחנו נעדכן את `dest`. מכיוון ש- `dest` הוא בעצמו איבר במערך, אנחנו משנים את הערכים של המערך **תוך כדי תנועה**. החישוב ישתנה!
- **בגרסה ה-buffer:** הערך של `tot` נשמר בצד ברגיסטר. המערך `vec` לא משתנה תוך כדי הלולאה, ורק בסוף נכתוב ל- `dest`.

- אופטימיזציה של הקומפיילר, שתצור לולאה אינסופית:

```
int main() {
    char buf[50] = "y";
    for (int j = 0; j < 9; j++) {
        printf("%d\n", j * 0x20000001);
        if (buf[0] == 'x') break;
    }
}
```

```
for (j = 0; j < 9 * 0x20000001; j += 0x20000001) {
    printf("%d\n", j);
    if (buf[0] == 'x') break;
}
```

אבל בגלל ש $Max_Int > 1 \times 2000000 \times 9$ אזי זה יוחלף ב true, והרי לולאה אינסופית. בעיה.

רשימת אופטימיזציות (רעיוניות, ממבחנים, ומהמצגות):

1. $A[n][j] \rightarrow A[n][j]$
2. לחסוך בחישובי כפל וכו', או חילוק. כיצד נמנע מכפל? נחשוב על חיבור.
3. Loop Unrolling (כלל אצבע: נפתח לרוב 4 איטרציות)
4. נחלק ל-2 סכומים על מנת שלא לפגוע ב Pipeline וכו'.
5. בלולאה עדיף לבדוק השוואה של $j \geq 0$ ולקדם את הלולאה באופן יורד כלומר $j--$. יותר קל לבצע השוואה לאפס למעבד (באמצעות דגלים)
6. Inline – החלפת גוף הפונקציה בפונקציה עצמה (טוב כאשר הפונקציה קטנה)
7. שינוי סדר הפקודות על מנת למנוע stall's
8. Alignment – שמירת נתונים בכתובות שמתחלקות ב-8 או ב-16 בייטים – מאפשר למעבד לבצע זאת הרבה יותר מהר.

9. זה לא משנה אם הגדרנו בתוך הלולאה $int\ i=0$ או הגדרנו אותו מחוצה לה, זו אותה יעילות.

10. קטע קוד הבא: b יעיל יותר, כי מוקצה במחסנית ללא system call:

```
int * p = malloc(10*sizeof(int));
```

```
int p [10];
```

11. קטע קוד הבא: a יעיל יותר, בגלל שלא צריך להקצות אותו מחדש בכל איטרציה (על המחסנית).

```
int x;  
for(int i=0; i<10; i++) {  
    x = i * 2;  
    printf("%d", x);  
}
```

12. לשים לב – לקרוא כמה פעמים ל printf לעומת לקרוא פעם אחת הרבה פחות יעיל! כי כל קריאה זה system call חדש שעולה הרבה.

```
for(int i=0; i<10; i++) {  
    int x = i * 2;  
    printf("%d", x);  
}
```

13. הקצאה ב bss: נתבונן על הדוגמה משמאל. בגרסה הראשונה, a מוגדר אך לא מאותחל. לכן נשמר ב bss. המערכת לא צריכה לשמור את ערכו בקובץ ההרצה, אלא רק לזכור כי יש להקצות עבורם מקום של אפסים. לעומת זאת, בגרסה השנייה הוא כן מאותחל, ולכן הוא נשמר ב data, כלומר – ההבדל הוא שבגרסה השנייה הערך עצמו נשמר, ובראשונה לא. ונשמר = לוקח מקום.

a

```
int i;  
void main() {  
    i = 5;  
}
```

b

```
int i = 4;  
void main(){  
    i = 5;  
}
```

14. גישות לזכרון – אם ניתן להמנע מגישה לזכרון, זה יעיל יותר. כי לגשת לזכרון זה כמובן יקר.

15. Branch prediction: לשים לב – מבחינת פייפליין ברור כי b יעיל יותר כי לא צריך לנחש ולבצע פעולות יקרות.

a

```
int cnt = 0;
for (int i = 0; i < N; ++i)
    if (arr[i] & 1)
        cnt++;
```

b

```
int cnt = 0;
for (int i = 0; i < N; ++i)
    cnt += arr[i] & 1;
```

```
// N is a positive integer
#define N ...
int a[N], b[N], cnt_a=0, cnt_b=0, i;
for(i=0; i<N; i++) {
    cnt_a += a[i];
    cnt_b += b[i];
}
```

```
// N is the same positive integer
#define N ...
for(i=0; i<N; i++)
    cnt_a += a[i];
for(i=1; i<N; i++)
    cnt_b += b[i];
```

16. דוגמה חשובה: כאן לא ניתן להכריע! זו דוגמה קלאסית ליעילות של זכרון לעומת לולאת זמן הריצה ($2N$ לעומת N) למה?

אם N גדול – הקוד השני יעיל יותר, בגלל הקאש. (נתונים במיקום שונה בזכרון) – נשים לב כי המערכים יתכן מאוד אם הוא אכן גדול לא יצליחו להכנס יחדיו לקאש. אם N קטן – הקוד הראשון יעיל יותר, כי לא צריך להריץ את הלולאה פעמיים (והמערכים נכנסים שניהם כנראה לקאש).

g++

```
// N is a positive integer
#define N ...
typedef struct A {
    int x;
}A;
A array[N];

void main() {
    for(int i=0; i<N; i++)
        printf(array[i].x);
}
```

b

```
// N is the same positive integer
#define N ...
int array[N];

void main() {
    for(int i=0; i<N; i++)
        printf(array[i]);
}
```

17. דוגמה על struct's וכו': לשים לב! הקודים שקולים מבחינת יעילות והסיבה היא המעניינת – בגלל שבסטארקט יש איבר אחד בדיוק, מבחינת זכרון המערך הוא רצף של יחידות סטרקט int לעומת מערך של יחידות int - שקול לחלוטין.

אבל לשים לב - אם היה בstruct עוד שדות! המעבד היה צריך לקפוץ בייטים בין איבר לאיבר בסטרקט, למשל: אם היה לנו סטרקט שמכיל שדה של שם ושדה של ציון, והיה מערך של סטרקטים. לעומת זאת – שני מערכים: אחד של שמות, אחד של ציונים. עדיף להשתמש בשני מערכים לסכימה. מדוע? כשנביא שורה מהקאש, יכנסו יותר נתונים ("ציונים") בתוך שורת קאש במקרה השני, שכן במקרה הראשון נביא גם את השמות – שבכלל לא רלוונטים עבורנו. כלומר: זה יהיה טוב רק כאשר באמת נזדקק לכל המידע שמביאים.

18. Macro & inline: גם כאן, לא ניתן להכריע.

המאקרו הרי מטופל עוד בשלב הראשוני, ומוחלף בגרסה שלו. inline הוא רמז לקומפיילר – כלומר, זו בקשה לקומפיילר להחליף את הקריאה לפונקציה בגוף שלה. אך: מדובר בבקשה: יתכן והקומפיילר יתעלם מהבקשה! ולכן אם הקומפיילר לא יענה לבקשה, תתבצע קריאה לפונקציה שזה דבר יקר ולכן a יעיל יותר. אם הוא כן יענה, מדובר בקודים שקולים.

```
#define SQUARE(x) ((x)*(x))
void main() {
    int x = 4;
    x = SQUARE(x);
}
```

```
inline int square(int x) {
    return x*x;
}
void main() {
    int x = 4;
    x = square(x);
}
```

```
int f()
{
    int x = rand() % 2;
    int y = rand() % 10;
    if(x || y)
        return 1;
    return 0;
}
```

```
int f()
{
    int x = rand() % 2;
    int y = rand() % 10;
    if(y || x)
        return 1;
    return 0;
}
```

19. כעת מדובר בניתוח הסתברותי. נשים לב שכפי שאנחנו יודעים | ("או") לא יקרא את שני הנתונים אם הראשון הוא אחד. $Pr[X=0]=0.5, Pr[Y=0]=0.1$, הסיכוי של וואי להיות אחד גבוה הרבה יותר, ולכן קוד השני יעיל יותר.

20. לשים לב – פעולת and מהירה הרבה יותר מפעולות כמו sub, add וכו'. בכללי לוגיות עדיפות.

21. אופטימיזציה טובה היא מעבר לפוינטרים, ולהשוות בכל שלב את פוינטר ההתחלה לפוינטר הסיום (על מערך נניח), כך לא צריך לבצע פעולת ++ של ערך i בלולאה.

22. Duff Device: כשפותחים לולאת אנרלוינג, יש בעיה של שארית.

למעשה, הסוויץ' קיים משמאל פותר בדיוק את בעיית השארית הזו. הוא מבצע שמיניות שלמות, עד שנותרו לו פחות מ-8 איברים והוא מבצע את הקיים של השארית שנותרה לו. למה זו אופטימיזציה? נשים לב שבדקים תנאי של גדול ו++ רק כל 8 איברים, זה חוסך הרבה cmp, decrease וכו'. זה גם נוח לפייפליין כי כאן המעבד רץ על קוד לינארי ארוך לעומת קפיצה בכל איטרציה.

```
int compute_total_duff(int* vec, int len) {
    int tot = 0;
    int n = (len + 7) / 8;

    switch (len % 8) {
        case 0: do{ tot += *vec++;
        case 7:   tot += *vec++;
        case 6:   tot += *vec++;
        case 5:   tot += *vec++;
        case 4:   tot += *vec++;
        case 3:   tot += *vec++;
        case 2:   tot += *vec++;
        case 1:   tot += *vec++;
        } while(--n > 0);
    }

    return tot;
}
```

23. לולאת for ולולאת while (תחת ההנחה שהקוד שקול) שקולות לחלוטין מבחינת זמן ריצה – מבצעות את אותם

הדברים בדיוק: cmp, jmp, decrease

24. האופרטור הטרנארי ? מתורגם לרוב לרוב cmov וממש מהיר

25. לשים לב בדוגמה משמאל – קטעי הקוד שקולים, כיוון שממילא יש לבדוק את שניהם. הוא בודק $X > 0$, אם הוא צדק הוא בודק $y > 0$, אחרת הוא לא בודק. זה בדיוק מה שיש בקוד B.

B:

```
C
if (x > 0) { if (y > 0) { ... } }
```

```
C
int a[100]; // Global
for(i=0; i<100; i++) a[i] = 0;
```

B:

```
C
void f() {
    int a[100]; // Local
    for(i=0; i<100; i++) a[i] = 0;
}
```

26. A יעיל יותר, כתובת של מערך גלובלי קבועה וידועה כבר בזמן לינקינג. ולכן המעבד פשוט כותב את הכתובת הישירה בקוד. כלומר – המעבד לא עושה "חשבון" על מנת למצוא את תחילת המערך. במערך לוקלי – נוצר על המחסנית, הכתובת שלו משתנה בכל פעם והמעבד חייב לקחת את הערך של `rsp` ולהחסיר ממנו ערך ולכן בכל גישה למערך המעבד מבצע פעולות חיבור/חיסור נוספות שעולות כמובן.

27. פעולת `xor` קצרה יותר בבייטים (פיזית האורך שלה קצר יותר) `mov` למשל וכו', ולכן היא יותר זולה לשימוש.

A:

```
C
char a = 1; char b = 2; char c = a + b;
```

B:

```
C
int a = 1; int b = 2; int c = a + b;
```

28. לשים לב – קוד בי הרבה יותר מהיר!!! המעבד "חושב" ב-64 או 32 ביט. כשמבקשים ממנו לחבר שני צ'ארים הוא לא באמת יודע לחבר רק בייט אחד בודד, הוא קורא את הבייט, מרחיב סימן (הופך אותו למס' של 64/32 ביט) מבצע את החיבור, ואז לחתוך בחזרה ל-8 ביט (בייט). למעשה זו alignment – גישה לנתון שאיננו מיושר או קטן מדי מאלצת את המעבד לבצע פעולות לוגיות נוספות על מנת לבדוד את הנתון מתוך המילה המלאה. שימוש ב-`int` חוסך כל זאת.

29. אתחול יעיל למערך `int A[N]={0}`

30. שינוי סדר של פונקציות

31. אם יש לנו שתי פקודות די דומות – זו שהקידוד שלה לשפת מכונה קצר יותר תקח פחות זמן בשלב `fetch`

32. חישוב אריתמטי על קבועים מבוצע כבר בזמן קומפילציה ולא בזמן הריצה.

33. ביצוע `shift right` למספר `unsigned` מהיר יותר מביצוע `shift right` למספר `signed` שכן לא צריך לשמר את ה-`MSB`. לכן גם חילוק ב-2 של מספר `unsigned` מהיר יותר.

System Calls

- Unix files – מתייחס לעקרון לפיו "הכל הוא קובץ". כמעט כל דבר נחשב לקובץ -ספריות, ערוצי תקשורת, התקני חומרה וכו'. כשפותחים קובץ, מערכת ההפעלה לא נותנת את הקובץ עצמו, אלא מספר שלם קטן שמשמש כמזהה ייחודי עבור התהליך שלנו. למספר זה קוראים `file descriptor (fd)`

לכל תהליך שנוצר ב-Unix ישנם אוטומטית 3 קבצים פתוחים:

1. `fd=0: Input-stdin`, לרוב מדובר במקלדת.

2. Output-stdout: fd=1, לרוב זהו המסך.

3. Error-stderr: fd=2, הודעת שגיאה דחופה למסך.

כאשר נרצה לפתוח קובץ, נרצה לתת לו הרשאות של מה אנחנו יכולים לעשות זאת. את ההרשאה נכתוב כפרמטר השני של הפונקציה fopen. לדוגמה: `fp = fopen("my_notes.txt", "w");`. בטבלה למטה מצורף הסבר על ההרשאות השונות:

Mode	הרשאה	התנהגות אם הקובץ קיים	התנהגות אם הקובץ לא קיים
"r"	קריאה בלבד	פותח כרגיל מהתחלה	שגיאה (מחזיר NULL)
"w"	כתיבה בלבד	דורס (מוחק) את תוכן הקובץ	יוצר קובץ חדש
"a"	כתיבה (Append)	מוסיף לסוף הקובץ	יוצר קובץ חדש
"+r"	קריאה + כתיבה	פותח מהתחלה (לא מוחק)	שגיאה
"+w"	קריאה + כתיבה	דורס (מוחק) את התוכן	יוצר קובץ חדש
"+a"	קריאה + כתיבה	קורא מהתחלה, כותב לסוף	יוצר קובץ חדש

מלכודת פוטנציאלית: בהינתן קוד כזה,

```
;FILE *f = fopen("test.txt", "r")
```

```
;fprintf(f, "Hello")
```

האם הקוד יתקמפל? לא! אנחנו סגרנו את הרשאות הקובץ אפשרנו קריאה בלבד, אי אפשר להדפיס כי הדפסה זה כתיבה למסך!

קריאה: נכנסים נתונים שזורמים מהקובץ אל תוך הזכרון של התוכנית. כתיבה: נתונים יוצאים מהתוכנית החוצה אל הקובץ.

ברמת האסמבלי – הוא יתרגם את ההרשאות האלו ל"מילון" מיוחד של צירוף ביטים –

"r"=O_RDONLY

"w"=O_WRONLY|O_CREAT|O_TRUNC

"+r"=O_RDWR

- ההבדל בין משפחת הפונקציות read למשפחת הפונקציות fread:

1. Read – open,close,read,write וכו'. אלו פקודות שמדברות ישירות עם הkernel של linux. חיות קרוב לחומרה, כל קריאה לאחת מהן היא פנייה למערכת ההפעלה, המעבד עובר לקרנל מוד, מבצע את הפקודה ואחכ מחזיר את השליטה. איטי יחסית.
2. Fread – עטיפות חכמות שנכתבו עבורנו בC, הן חיות בתוך התוכנית שלנו, משתמשות בBuffer (זכרון זמני), כשקוראים fread על מנת לקבל תו אחד הספרייה מבצעת read אחד גדול מהקרנל (מביא הרבה מידע) ושומר אותה בbuffer שלה. בפעם הבאה שנבקש תו, היא תביא אותו מהזכרון המהיר שלה ללא להציק לקרנל. יותר מהיר לעבודה עם קבצי טקסט ונתונים קטנים. עובדות עם IO סטרים.
- לשים לב – הפונקציה write מחזירה כמה תווים היא כתבה. גם n/ הוא תו שצריך להספר שנכתב!
- O644 – כשיוצרים קובץ חדש עם open צריך להגיד למערכת ההפעלה – מי מורשה לגעת בקובץ זה? 0644 הוא הדרך לומר זאת בשיטה הoctal, הספרה הראשונה מציינת מס' אוקטאלי, ואז:
 1. 6 – הרשאות של הבעלים (אני)
 2. 4 – הרשאות של הקבוצה
 3. 4 – הרשאות של כל השאר

כל ספרה היא סכום של 3 ביטים, 4 מציין קריאה, 2 מציין כתיבה, 1 מציין הרצה. לכן $4+2+6=6$ כלומר מותר לבעלים לקרוא ולכתוב לקובץ. לקבוצה (כל מי ששווייך לקובץ), ולכל השאר (כל שאר האנשים עם גישה למחשב) – מותר רק לקרוא.

- O_TRUNC – כשפותחים קובץ עם הדגל אומרים למערכת ההפעלה: אם הקובץ הזה כבר קיים, תמחקי את כל התוכן שלו ותהפכי אותו לקובץ ריק (גודל 0) ברגע הפתיחה. בלי דגל זה – כתיבה תשמור על מה שהיה ותכתוב בסוף/תעלה על הקיים.

- Lseek - נחשוב על קובץ כסרט מגנטי ארוך. בכל פעם שנקרא או נכתוב תו, הראש של המגנט זז קדימה. Lseek מאפשרת להזיז את הראש לאן שבא לנו בתוך הקובץ מבלי לקרוא אף תו. כך נראה הסינטקס:

- `off_t lseek(int fd, off_t offset, int whence);`

נשים לב כי בהרצאה `offset=0` היא יכולה לבלבל ושהוא לא יהיה אפס. הפרמטר הכי חשוב הוא `whence` ויש לו 3 אפשרויות:

1. `SEEK_SET`: `offset` נספר מתחילת הקובץ לפי `offset`, לדוגמה:

`lseek(fd, 10, SEEK_SET)` – כאן יציבו את הראש בדיוק בבייט ה-10 מתחילת הקובץ.

2. `SEEK_CUR` – `offset` נספר באופן יחסי למקום שאנו נמצאים בו כרגע.

3. `SEEK_END` – `offset` נספר יחסית לסוף הקובץ.

System calls: מדובר ב"דלת" היחידה שדרכה התוכנית שלנו יכולה לבקש מהחומרה לבצע משהו. כשצריך לבצע פקודה – מבקשים ממערכת ההפעלה, היא שמה את הנתונים ברגיסטרים ספציפיים, היא קופצת (מבצעת את הפקודה `syscall`, המעבד עובר לKernal Mode, מערכת ההפעלה בודקת שיש הרשאות, מבצעת את העבודה וחוזרת לתוכנית עם התוצאה.

קריאות מערכת שלמדנו (מופיע בדף נוסחאות, לכן נדגים אחת):

1. `Read`:

מספר הSystem Call: 0

`Rdi`: מציין את `file descriptor`.

`Rsi`: מדובר בפוינטר אל `buffer` ממנו נקרא.

`Rdx`: כמה בתים עלינו לקרוא.

```
.section .bss
buffer: .space 1

.section .text
.globl _start
_start:

movq $0, %rax    # system call number (sys_read)
movq $0, %rdi    # file descriptor (stdin)
leaq buffer(%rip), %rsi    # buffer to keep the read data
movq $1, %rdx    # bytes to read
syscall          # call kernel

movq $60, %rax   # system call number (sys_exit)
movq $0, %rdi    # exit status
syscall          # call kernel
```

ערך החזרה מכל System :call

שם הפונקציה	מספר Syscall (%rax)	ארגומנטים מרכזיים	ערך חזרה בהצלחה (ב- %rax)	ערך חזרה בכישלון
read	0	fd, buf, count	מספר הבייטים שבאמת נכתבו לזיכרון	1-
write	1	fd, buf, count	מספר הבייטים שבאמת נשלחו ליעד	1-
open	2	filename, flags, mode	ה-File Descriptor החדש (מספר שלם חיובי)	1-
close	3	fd	0	1-
lseek	8	fd, offset, whence	המיקום החדש (בבייטים) מתחילת הקובץ	1-
fork	57	אין	לאבא: ה-PID של הבן. לבן: 0.	1-
execve	59	path, argv, envp	לא חוזר לעולם (התוכנית מוחלפת)	1-
exit	60	error_code	לא חוזר (התהליך מסתיים)	-

- Ascii: נתבונן בקוד הבא-

```
msg: .ascii "Hello\n"
```

```
len = . - msg
```

מדובר בהוראה לאסמבלר – לשמור את התווים בזכרון זה אחר זה. נבחין כי פקודה זו לא מוסיפה 0 \ באופן אוטומטי! בשורה הבאה, במעין טריק מחושב אורך המחרוזת, האסמבלי יחשב ואם נעשה \$len נחזיק את אורך המחרוזת.

- Unix Process: process הוא תוכנית שנמצאת בפעולה. כשאנחנו מריצים תוכנית, אנחנו מבקשים ממערכת ההפעלה לקחת את הקובץ שלנו ולהריץ אותו. ואז, מערכת ההפעלה מבצעת process image ובונה process image.
 - **Fork** – קריאת מערכת, שיוצרת child process.
- ברגע שהמעבד מריץ פקודה fork, מערכת ההפעלה יוצרת העתק מדוייק של התהליך הקיים. הבן מקבל העתק מדוייק של כל הזכרון (משתנים, מחסנית, heap וכו'). מה שונה בניהם? ערך החזרה מהפונקציה שמבדל בניהם. מקבל העתק של כל fd ושני התהליכים ממשיכים לרוץ בדיוק מאותה נקודה.
- מה בדיוק זה בניהם?** המצביע למיקום הנוכחי בקובץ של קבצים פתוחים (הבן מקבל העתק של טבלת FD אך שני DF's מצביעים לאותו open file descriptor בקרנל – בגלל שהטבלה היא בקרנל, זה משותף!!!).
- אך יודעים מי זה מי? לפי הרגיסטר rax שמחזר כתוצאה מהsyscall. בתהליך הבן, fork יחזיר 0 ובתהליך האב fork יחזיר את pid (ת"ז של הבן) – מס' חיובי שלם גדול ממש מאפס. אם חזר מס' שלילי – השכפול נכשל (אין מקום).
- לשים לב – טבלת fd של התהליכים של האבא והבן משוכפלת, אם בן סגר קובץ אבא עדיין יכול לכתוב לתוכו.
 - אין שום הבטחה מי ירוץ קודם ואי אפשר לדעת זאת. (מבין האבא והבן). זה תלוי במערכת ההפעלה.
 - Wait – כשתהליך בן מסיים את תפקידו, הוא לא נעלם לגמרי מהמערכת. הוא הופך לזומבי: הוא מת, אך הנתונים שלו עדיין תופסים שורה בטבלת התהליכים של הקרנל. כשאבא קורא לwait קורים 3 דברים:
 1. המתנה – אם הבן עדיין רץ, האבא עוצר, ומחכה עד שהבן יסיים.
 2. קבלת מידע – האב מקבל את ה"מילים האחרונות" של הבן, הקוד שהוא שלח בexit.
 3. ניקוי: ה kernel מוחק סופית את הבן מטבלת התהליכים.
- Wait(&status) – דרך של האבא לקבל "דוח פטירה" מהבן. מביאים לkernel כתובת של משתנה שאצלנו בזכרון, כשהבן מת הkernel דוחף לתוך הכתובת הזו מספר שמכיל את "סיבת המוות", כדי לקרוא את המידע משתמשים בכמה מקרואים שלא נפרט יותר מדי.
- תהליך זומבי לא תופס עוד מקום בזכרון אך נשמר בטבלת התהליכים על מנת שאבא שלו יוכל לראות מדוע הוא מת.

- אם לאבא יש 1GB זכרון בשימוש, בעת יצירת הבן לא יוקצה מיד עוד 1GB עבור הבן, אלא יהיה דף זכרון משותף עד לרגע שבו אחד התהליכים מבצע כתיבה – זה אופטימיזציה.

- **Execvp**: מחליף את הקוד של התהליך. דורס את התוכנית הנוכחית בזכרון בתוכנית חדשה. ה-PID נשאר זהה! הפרוסס לא משתנה – רק תוכנו מוחלף. ערך החזרה: בהצלחה לעולם לא יחזור (כי התחלף הרי) ובכשלון -1. להלן הסינטקס:

```
int execvp(const char *file, char *const argv[]);
```

- file – שם קובץ להרצה חדש, argv הוא מערך של מצביעים למחרוזות (התוכנית עצמה, דגלים וכו', האיבר האחרון במערך חייב להיות null). מדוע צריך את מערך המצביעים למחרוזות? ממש כמו במain שאנחנו מכירים.
- מה ההבדל בין exec,execvp? הק בסיום גורמת למערכת לחפש את הקובץ אוטומטית בנתיבים המוגדרים במשתנה הסביבה PATH.
- System call Tabel: מדובר במערך של מצביעים לפונקציות, שנמצא בתוך הקרנל ספייס. כל תא במערך מצביע לקוד שמבצע פעולה ספציפית (כמו read,write וכו'). כיצד זה עובד? המערכת משתמשת במספר ששמת בrax כאינדקס (מיקום בתוך הטבלה). הקפיצה – כשמריצים syscall המעבד עובר לקרנל מוד וקופץ ישירות לכתובת שרשומה בטבלה באותו אינדקס. מדוע זה ככה? למען אבטחה – אי אפשר להריץ קוד של קרנל ישירות, אפשר לבקש מס' מהטבלה וה-OS מחליטה אם ואיך לבצע.

- Shell מבצע fork לפני שהוא קורא ל-execvp() כיוון שהוא רוצה למנוע מ-execvp() לדרוס את תהליך ה-shell עצמו ולסיים אותו. כלומר: אם shell היה קורא ל-exec ישירות כדי להריץ פקודה ls אזי ה-shell היה נעלם מהזכרון, ls היה רץ במקומו, כשls היה מסיים לא היה מי שיחזור למסך – המחשב פשוט היה נתקע כי התהליך של shell היה נמחק.
- המספר 0 שמוחזר בrax לאחר sys_read מייצג את EOF: כאשר המצביע נמצא בסוף הקובץ ואין יותר מה לקרוא, sys_read מחזירה 0 בתים שנקראו.
- אם יש לנו fork() fork() fork() fork() וכו'.... כאשר n הוא מס' הפעמים שכתוב פורק, אזי יפתחו 2^n תהליכים.
- אם לתהליך אין ילדים בנים, הפונקציה wait תחזיר -1.
- PID מזהה תהליך, PPID מזהה של הורה של תהליך.

• Macro's

מאקרו היא תבנית שמוחלפת בקוד המקורי כבר בשלב pre processing.

1. עלינו לשים לב שמאקרו מתרחב ממש כהעתק, ולכן לא תמיד יתבצע בדיוק מה שנרצה. למשל:

```
#define badExample(a) a*a
badExample(5+1)
→ 5+1*5+1=5+5+1=11 😞
```

2. לשים לב שמאקרו לא עובד עם גרשיים. כלומר,

```
#define Macro(a) "a"
Macro(10)
→ "a"
```

כלומר, הוא לא מחליף את מה שכתוב ומקשר אלא ממש מתייחס לזה כמחרוזת טקסט.

3. האופרטור #: מעתיק את המחרוזת כמחרוזת as is. דוגמה:

```
#define TO_STRING(x) #x
printf("%s", TO_STRING(10 + 5));
→ 10+5 //!15 לא מדפיס
```

4. האופרטור ##: משמש להדבקה של שני טוקונים יחד כדי ליצור אסימון אחד. חשוב: רווחים נעלמים! דוגמה:

```
int grade_math = 95;
int grade_physics = 100;
#define GET_GRADE(subject) grade_## subject
printf("%d", GET_GRADE(math));
```

כיוון שהם יוצמדו, השורה תהפוך לגרייד_מאט ולכן יודפס 95.

5. דוגמה חשובה:

מה קורה בשורה הראשונה? כשכותבים STR(FOO) pre-processor רואה את s# ומבצע פעולה פשוטה – מחליף את s ב-FOO. ולכן, זה נותר FOO בלבד ללא להגיע לשלב שבדקים שהוא אכן 4.

מה קורה בשורה השניה? כאן משתמשים במאקרו ביניים, למה זה כן עובד? כשPreProcessor פוגש את XSTR(FOO) מתרחש התהליך הבא:

1. הוא רואה כי XSTR מאקרו ומסתכל על הפרמטרים שלו

2. SSTR איננו משתמש ב# או ## ולכן הוא קודם כל מרחיב את הפרמטרים, עכשיו הביטוי הוא STR(4), זה גם מאקרו, והוא מרחיב אותו.

ולסיכום: בשני קראנו למאקרו נוסף, ובראשון לא ובפרט בגלל שהוא רואה # הוא מעתיק את זה אס איז.

```
#define STR(s) #s
#define XSTR(s) STR(s)
#define FOO 4

STR(FOO);
XSTR(FOO);
```

STR(FOO) -> #FOO -> "FOO"

XSTR(FOO) -> XSTR(4) -> STR(4) -> #4 -> "4"

```
"FOO";
"4";
```

כלל: ברגע ש-preprocessing רואה # או ## הוא מפסיק לבדוק אם זה מאקרו.

6. מספר משתנה של ארגומנטים: כמו פונקציה, גם מאקרו יכול לקבל מס' משתנה של ארגומנטים. שמים ... (3)

נקודות) בסוגריים של הדפיין על מנת להבהיר כי הפונקציה תקבל כמה משתנים, ושמים __va_args__ שזהו מזהה מיוחד שמוחלף בכל הארגומנטים שהמשתנה שולח. דוגמה:

```
#define eprintf(...) fprintf(stderr, __VA_ARGS__)
```

7. מאקרואים אינם רקורסיביים! דוגמה:

```
#define FACT(n) (n) * FACT(n-1)
```

הגיונית, ציפינו כי אם נבצע fact(10) המאקרו יתרחב ויחזיר 10!, אך לא. ה-preprocessing מזהה את fact, הוא מחליף אותו בביטוי 10*fact(10-1), הוא מסתכל על התוצאה ורואה שוב fact אך בגלל שהוא כבר בתוך הרחבה הוא מסמן אותה ככחולה ולא נוגע בה. התוצאה הסופית תהיה הטקסט 10*fact(10-1), הקומפיילר ינסה לקמפל את זה, יחפש פונקציה בשם fact שאיננה קיימת, ונקבל שגיאת לינקוף. (יתכן ואף שגיאת קומפילציה, זה לא מובטח).

סיכום כללים:

כאשר קוראים למאקרו רגיל ללא # ## הוא:

1. מרחיב את כל הארגומנטים ששלחנו לו

2. שותל את התוצאות המורחבות בגוף המאקרו

3. סורק שוב את התוצאה הסופית כדי לראות אם נוצר משהו חדש שיש להרחיב.

ברגע שארגומנט מופיע ליד # או ##, הוא מדלג אוטומטית על שלב 1.

הבהרה: הוא יבצע שוב ושוב סריקה חוזרת עד שלא יוכל להתקדם (לא יהיה מאקרו). זה לא רקורסיה, למה? כי מתקדמים למקרואים הבאים, זה נחשב רקורסיה (ולא חוקי וזה עוצר) אם חוזרים לאותו משתנה. לדוגמה:

```
#define THREE 3
#define TWO THREE
#define ONE TWO
```

```
int x = ONE;
```

```
→One→Two→Threee→3
```

- לשים לב טוקן של ab ## 12 יניב שגיאת קומפילציה כי זה הרי 12ab ומשתנה לא יכול להתחיל במספר לפי חוקי C.

:Inline Assembly

סינטקס נראה ככה:

```
asm ( "code" : outputs : inputs : clobbers );
```

משמעות של כל חלק:

חלק	שם	תפקיד
"code"	Assembler Template	מחרוזת של פקודות אסמבלי (כמו <code>movq , addl</code>). משתמשים ב-%0, %1 וכו' כדי לייצג משתנים, וב-% כדי לייצג רגיסטרים (למשל <code>rax%%</code>).
outputs	Output Operands	רשימת משתני C שיקבלו ערך. חייבים להתחיל ב- = (מסמן כתיבה). האופרנד הראשון ברשימה יהיה %0, השני %1 וכן הלאה.
inputs	Input Operands	רשימת משתני C שהאסמבלי קורא מהם. הם ממשיכים את הספירה של ה- outputs (למשל, אם היו 2 פלטים, הקלט הראשון יהיה %2).
clobbers	Clobber List	רשימת רגיסטרים שהקוד שלך משנה "על דעת עצמו" ולא דרך ה- outputs. זה אומר לקומפיילר: "אל תשמור פה מידע חשוב, אני הולך לדרוס את הרגיסטר הזה".

Constraint	Meaning
"=r"	Update value stored in a register
"+r"	Read and update value stored in a register
"=m"	Update value stored in memory
"+m"	Read and update value stored in memory
"=rm"	Update value stored in a register or in memory
"+rm"	Read and update value stored in a register or in memory

:Shell

Shell הוא ממשק בין המשתמש למערכת ההפעלה.

- ישנם כמה סוגים של shell – Bash, Zsh. על מנת לבדוק באיזה מהם אנחנו נריץ `echo $SHELL`.
- משתני סביבה: משתני סביבה הם משתנים שמשפיעים על התנהגות shell ועל תהליכים אחרים במערכת. בניגוד למשתני של רגילים שמוגדרים רק לשימוש פנימי, משתני סביבה זמינים גם לתוכניות וסקריפטים שמריצים מתוך השל.
 1. PATH – מציג רשימת נתיבים paths שבהם shell מחפש קבצי הרצה כאשר אתה מקליד פקודה.
 2. PWD – מכיל את הנתיב של תיקיית העבודה הנוכחית שלך
 3. SHELL – הנתיב לתוכנית השל שבה אתה משתמש כרגע
 4. USER – שם המשתמש שמחובר כרגע למערכת
 5. HOME – נתיב לתיקיית בית של יוזר
 - 6.
- פקודות שכדאי להכיר:
 1. Printenv – הפקודה להצגת כל משתני הסביבה (ורק משתני הסביבה!!!!) שמוגדרים כרגע
 2. Getenv() – מאפשרת לתוכנית לגשת לערך של משתנה סביבה כלשהו
 3. Export – פקודה שמשמשת להפיכת משתנה למשתנה סביבה. כשמגדירים משתנה רגיל הוא חי רק בתוך השל הנוכחי, תוכניות אחרות שנריץ ממנו לא יכירו אותו. הפקודה `export` עושה 2 דברים: 1. היא מייצאת את המשתמש כך שכל תהליך סקריפט או תוכנית שתפעיל מטרמינל זה יכירו אותו ויכולו להשתמש בערך שלו. 2. זמניות: זה תקף רק לטרמינל הנוכחי. אם תסגור ותפתח טרמינל חדש, הוא ייעלם.

- כיצד ניתן ליצור משתני shell? לדוגמה: VAR="value" ובשורה לאחר מכן בודקים (ומדפיסים) אותו לבדוק שאכן נוצר echo \$VAR
- ההבדל בין משתנה shell למשתנה סביבה – משתנה של הוא משתנה מקומי, תוכניות לא יכירו אותו. משתני סביבה הם משתנים גלובליים, כל תוכנית שתצא מהשל הזה תוכל לראות אותו.
- Chmod – פקודה בסיסית ביוניק שמשמשת לשינוי הרשאות הגישה לקבצים ולתיקיות. כשאתה יוצר קובץ טקסט חדש עם פקודות Bash, המערכת מתייחסת אליו כקובץ טקסט רגיל לקריאה/כתיבה בלבד. כדי שה Shell-יסכים להריץ את הקובץ כתוכנית, אתה חייב להעניק לו "הרשאת הרצה" – את זה עושים עם chmod.
- _start:
- מדובר בנקודת הכניסה Entry point של הקובץ הבינארי בפורמט ELF. כשמריצים תוכנית, מערכת ההפעלה טוענת אותה לזכרון וקופצת ישר לכתובת של _start. כשסטארט מתחיל לרוץ, אין עדיין סביבת עבודה מסודרת. אחריותו "להקים את התשתית". הוא עושה את הדברים הבאים:
 1. מקים את המחסנית
 2. מטפל בארגומנטים – מעביר את argc (מס' ארגומנטים) argv (מערך ארגומנטים) מהמערכת למקומות ש main מצפה למצוא אותם.
 3. אתחול משתנים גלובליים
- לאחר שכל ההכנות הסתיימו, _start מבצעת call לפונקציה main. סטארט מסופקת ע"י הלינקר (או מוגדרת בספריית C), ועוד main היא פונקציה שהמתכנת מגדיר!
- כשפונקציה main מסתיימת, היא חוזרת אל מי שקרא לה. אם לא היה את _start לא היה לאן לחזור והתוכנית הייתה קורסת. לכן,
- תפקיד הפונקציה בסיום: לקבל את ערך החזרה מ main, לבצע ניקוי וביצוע של exit system call על מנת לומר למערכת ההפעלה כי התהליך רשמית הסתיים.

למה שמתכנת ירצה ליצור פונקציית start בעצמו?

1. לצמצם את גודל הקובץ (במקורית יש הרבה פונקציות אתחול של libc)
 2. בניית תוכניות ללא תלות – כתיבה לבד שלא מסתמכת על פונקציות ספרייה
 3. שליטה על האתחול – אתה יודע בעצמך מה עשית
- אפשר להוסיף דגל שאומר ללינקר לא להוסיף את _start כי נביא אחת כזו בעצמנו.
 - הלינקר אחראי להכניס את קוד ה_start לתוך הקובץ הבינארי הסופי וקובע אותו כנקודת הכניסה. Loader אחראי להעביר את השליטה בהתחלה אל _start.
 - Strace: כלי שמאפשר לראות את כל קריאות המערכת שהתוכנית שלי מבצעת בזמן אמת. אם נריץ strace על פקודה שלא קיימת השל יחזיר שגיאה עוד לפני שסטרייס יתחיל לרוץ.
 - Ls – תפקיד הפקודה להציג רשימה של קבצים ותיקיות שנמצאים בתוך התיקייה שאנו בה כרגע.
 - GPU: מעבד שתוכנן במיוחד לביצוע חישובים מקביליים פשוטים. cpu ממוצע יש 4 ליבות, gpu יש כ-16,000 ליבות. הוא מריץ הוראה אחת על הרבה מאוד נתונים במקביל.

משתני סביבה בc

ישנן 3 דרכים לגשת למשתני סביבה מתוך הקוד שלנו:

1. באמצעות הפונקציה `getenv`:
a. למשל:

```
1. char *val = getenv("User");
```

b. זו מחזירה מצביע למחרוזת אם המשתנה קיים, אחרת NULL.

c. חשוב – אסור לבצע `free` לזיכרון שמוחזר ע"י הפונקציה הזו, ואסור לשנות אותו ישירות.

2. המשתנה הגלובלי `environ`:

a. נוכל להגדיר אותו בתחילת התוכנית שלנו:

```
1. extern char **environ;
```

b. זהו מצביע למערך של מחרוזות, כאשר כל מחרוזת היא מהפורמט `KEY=VALUE`.

המערך הינו `Null-Terminated`.

3. הארגומנט השלישי של `main`:

a. לא הרבה יודעים זאת, אבל POSIX מאפשרים ל`main` להיראות כך:

```
1. int main(int argc, char *argv[], char *envp[]);
```

b. כאשר `envp` הינו למעשה עותק מקומי של `environ`.

Memory Errors (תרגול 12)

בשפה כמו פייתון, אם ננסה לכתוב מעבר למקום הזכרון שהוקצה לנו נקבל שגיאה. לעומת זאת, בשפה כמו c או אסמבלי, לא נקבל שגיאה. מדוע? כי סי רצתה להיות קרובה לברזלים – היא לא מבזבזת זמן על בדיקת גבולות בזמן ריצה ולמעשה סומכת (יתכן שיתר על המידה) על המתכנת. בעיה רצינית – היא אם ניגשנו למיקום במערך שלא מוגדר ומיקום זה יחושב ע"פ אדרסינג מוד, ויתכן ובמיקום זה יש את כתובת החזרה של הפונקציה. נכנס כאן להמון בעיות רציניות.

- **Buffer Overflow**: באפר הוא רצף של בתים בזכרון שהוקצו לשמירת נתונים, בסי זה ע"י מערך. מה הכוונה באוברפלו? גלישה קוראת כאשר מנסים לכתוב לתוך הבאפר יותר נתונים ממה שהוא יכול להכיל. כיוון שסי לא בודקת גבולות – היא פשוט ממשיכה לכתוב לכתובות הבאות בזכרון. גם אם הן שייכות למשהו אחר. הבעיה היא שהמידע הקריטי נמצא מעל הנתונים של המערך – ולכן יתכן שנדרוס את המידע הקריטי של הפונקציה לחזרה. ככה, בעת הכנסת נתונים, ניתן לבצע התקפה – להכניס קוד זדוני שידרוס את כתובת החזרה, וכשהפונקציה תבצע `ret` זה יחזור אל הקוד הזדוני.
- **Stack smashing**: "ניפוף המחסנית", תוקף מנצל פרצה של באפר אוברפלו כדי להרוס את המבנה התקין של המחסנית. המטרה הסופית היא לא סתם לגרום לתוכנית לקרוס אלא לשנות את כתובת החזרה ששומרת על המחסנית. ברגע שהיא משתנה, התוקף שולט לאן תלך התוכנית. נניח וכבר הצלחנו להשחית את הזכרון, מה נוכל להשיג?
 1. השחתת זכרון חסר חשיבות – דריסה של אזור בזכרון שלא קריטי לתוכנית או אפילו ריק. לא שימושי עבור התוקף.
 2. השחתת ערכים שמספיעה על לוגיקה – דריסה של משתנה פשוט שמספיע כיצד התוכנית תתנהג.
 3. דריסה של כתובת זכרון שהתוכנית מתכננת לקרוא ממנה מידע, זה מאפשר לתוקף להכריח את התוכנית להדפיס/להשתמש במידע שהיא לא אמורה לראות שאני שמתי לה.
 4. דריסה של כתובת זכרון שהתוכנית מתכננת לכתוב אליה מידע – הוא מאפשר לתוקף להכריח את התוכנית לכתוב נתונים במיקום לא טוב ובכך להשחית קבצי מערכת או ערכים קריטיים במקומות רחוקים בזכרון.
 5. דריסת קוד קריטי כמו ערך חזרה לפונקציה או מצביע לפונקציה – זה הכי נזק קריטי.
- **Stack Canaries**: מנגנון הגנה שנועד לזהות באפר אוברלפו בזמן אמת לפני שזה גורם נזק למסלול הריצה של התוכנית. שמים "קנרית" (ערך סודי) במקום אסטרטגי במחסנית כדי למות ברגע שיש ניסיון פריצה. כך זה עובד:
 1. הצבה בכניסה לפונקציה – כשהפונקציה מתחילה לרוץ, הקומפילר מוסיף קוד קצר שבוחר מס' אקראי (קנרית) ומניח אותו על המחסנית. מיקומו קריטי – יושב בין משתנים מקומיים לבין כתובת החזרה.
 2. אם מתרחשת באפר אוברפלו – התוקף חייב לעבור דרך הקנרית ולדרוס אותה בדרך.
 3. רגע לפני שהפונקציה מסתיימת (ריטרן), התוכנית מבצעת פקודת תקינות – היא משווה בין ערך הקנרית שנמצא כרגע לבין הקודם. אם הוא זהה הכל כרגיל, אם לא אזי התוכנית מבינה שקרה משהו שלא היה צריך לקרות, התוכנית "מתאבדת" מיד ומוציאה שגיאה `stack smashing detected`.

נשים לב כי הקנרית מגינה על כתיבה בלבד, ולא מגינה על קריאה. התוקף יכול לקרוא מהמחסנית מידע קריטי.

- ASLR: הקנרית ניסתה להגן על המחסנית מפני דריסה, הוא מנסה למנוע מהתוקף לדעת היכן הוא צריך לקפוץ. לפני aslr, זכרון התוכנית היה סטטי – בכל פעם שהרצת את התוכנית המחסנית הערימה והדטה היו נשמרים באותו מקום. היא משנה את חוקי המשחק – בכל פעם שהתוכנית עולה לזכרון מערכת ההפעלה מגרילה כתובות בסיס חדשות לכל האזורים סטאק, היפ, לייברייז וכו'. כעת, גם אם התוקף הצליח לבצע באפר אוברפלו ולדרוס את כתובת החזרה, הוא עומד בפני ניחוש בלתי אפשרי – להיכן לכוון אותה? התוקף לא יודע איפה נמצאת התוכנית שלו עצמו בזכרון שהוא רוצה לכוון אליה, ולכן גם אם הוא ינחש הסיכוי שיפגע לאותו מקום אפסי. במערכת 32 ביט אין מספיק אפשרויות לכתובות אקראיות והתוקף יכול לנסות ולנסות ניסוי וטעייה ובסוף להצליח, ב-64 ביט זה כמעט ובלתי אפשרי.
- NX: ביט מיוחד שנמצא בטבלת הזכרון של המעבד, תפקידו לסמן אזורים ספציפיים ככאלו שמותר לקרוא מהם נתונים או לכתוב אליהם אבל אסור בשום פנים ואופן לבצע פקודות מתוכם. סימון בזיכרון שאומר למעבד שבאזורים מסוימים (כמו המחסנית) יש רק נתונים (כמו מספרים או טקסט) ואין שם פקודות להרצה. איך זה מונע פריצה? גם אם תוקף הצליח להזריק "קוד זדוני" לתוך המחסנית, ברגע שהמעבד ינסה להריץ אותו הוא יזהה שזה אזור שאסור להריץ ממנו קוד ויעצור את התוכנית מיד. השורה התחתונה: המנגנון הזה מפריד פיזית בין המקום שבו שומרים מידע לבין המקום שבו מותר להריץ פקודות, מה שחוסם הרצה של קוד שהוזרק מבחוץ.

טריקים נוספים ולזכור חשוב מאוד:

- אם יש לנו קוד כ"ל:

Int x=-1

Unsigned int y=1

If x>y

ההשוואה תתבצע לפי אנסיינגד! כלומר איקס יומר לאנסיינגד. כיוון ש-1 יצור מס' עצום, נקבל אמת – התנהגות לא מוגדרת.

- כבר בשלב הfetch של פקודה הrip יעבור להצביע על השורה הבאה לביצוע. רגיסטרים יעודכנו בשלב write
- executen וזכרון בשלב
- פקודות mov,lea לא משפיעות על RFLAGS
- אפשר ליצור סקשן בעצמנו ולתת לו הרשאות בעצמנו
- בכל תוכנית שהיא תמיד יהיה קריאה לפונקציה, בפרט לsyscall exit ובפרט שימוש ברגיסטרים
-